

Disjoint Set Union (DSU) / Union Find (Union by Size)

Complete Engineering Documentation & Visual Execution Walkthrough

Data Structure

Graph Algorithms

Path Compression

Union by Size

$O(\alpha(N))$ Complexity

1. What is DSU?

Disjoint Set Union (DSU) is a data structure used to efficiently manage a collection of disjoint (non-overlapping) sets.

It supports two main operations:

- **Find** → Determine which set an element belongs to.
- **Union** → Merge two different sets into one.

It is widely used in graph algorithms for:

- Finding connected components
- Detecting cycles
- Kruskal's Minimum Spanning Tree
- Network connectivity problems
- Dynamic connectivity problems

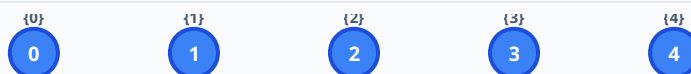
2. Basic Idea

Initially, every node belongs to its own set.

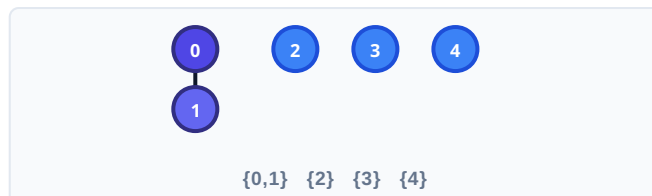
Example:

0 1 2 3 4

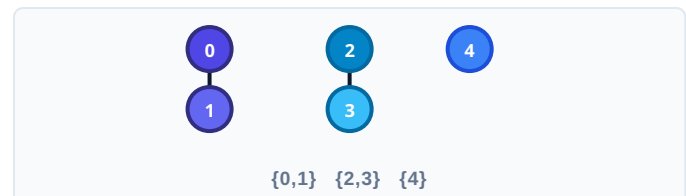
Each node is the leader of its own set.



After connecting 0 and 1:



After connecting 2 and 3:



The job of DSU is to maintain these groups efficiently.

3. Class Structure & Arrays

```
class DSU { public: vector<int> parent, size;
```

Explanation: Two arrays are maintained.

1. parent[]

Stores the parent of each node.

Initially:

Node	0	1	2	3	4
Parent	0	1	2	3	4

Every node is its own parent.

Meaning: 0 is leader of Set A, 1 is leader of Set B, 2 is leader of Set C...

2. size[]

Stores the number of nodes in the component whose leader is this node.

Initially:

Node	0	1	2	3	4
Size	1	1	1	1	1

Each component has exactly one node.

4. Constructor Implementation

```
DSU(int n) { parent.resize(n); size.resize(n, 1); for (int i = 0; i < n; i++) parent[i] = i; }
```

Step 1: `parent.resize(n);` — Creates a parent array of size `n`. Suppose `n = 5`. Memory becomes:

parent	—	—	—	—	—
--------	---	---	---	---	---

Step 2: `size.resize(n, 1);` — Creates size array:

size	1	1	1	1	1
------	---	---	---	---	---

Every node starts as an individual component.

Step 3: `for(int i = 0; i < n; i++) parent[i] = i;` — Makes every node its own parent.

Parent	0	1	2	3	4
--------	---	---	---	---	---

Graph representation:



5. Ultimate Parent & Path Compression: `findUPar()`

```
int findUPar(int node) { if (node == parent[node]) return node; return parent[node] = findUPar(parent[node]); }
```

Purpose: Returns the Ultimate Parent (Leader) of a node. The ultimate parent is the representative of the entire component.

Base Case

```
if (node == parent[node]) return node;
```

If a node is its own parent, then it is already the leader.

Example: parent `[0, 0, 1, 3]`

Calling `findUPar(3)`: Since `parent[3] == 3`, Return `3`.

Recursive Case (Path Compression)

```
return parent[node] = findUPar(parent[node]);
```

Example: Suppose chain `0 ← 1 ← 2 ← 3`

Parent array: `[0, 0, 1, 2]`

Call `findUPar(3)`

```
findUPar(3) ↓ findUPar(2) ↓ findUPar(1) ↓ findUPar(0)
↓ return 0
```

Without Path Compression vs After `parent[node] = findUPar(parent[node]);`



Why Path Compression?

Without it: `0 ← 1 ← 2 ← 3 ← 4 ← 5`. Finding `findUPar(5)` takes `5 → 4 → 3 → 2 → 1 → 0` which is $O(n)$.

After compression: Tree becomes flattened with root 0 and direct children 1, 2, 3, 4, 5. Now `findUPar(5)` takes only one step.

6. unionBySize() Implementation

```
void unionBySize(int u, int v) {
```

Purpose: Merge the components containing nodes `u` and `v`. Notice: It merges components, not necessarily the two nodes directly.

Step 1: Find Leaders

```
int pu = findUPar(u); int pv = findUPar(v);
```

Find the leaders of both components. Suppose Component 1 is `0 ← 1 ← 2` and Component 2 is `3 ← 4`. Calling `unionBySize(2,4)` gives `pu = 0`, `pv = 3` (leaders).

Why find leaders first?

Suppose we directly did `parent[u] = v;` (`2 → 4`). The DSU structure would become inconsistent because 2 and 4 may not be the leaders of their components. DSU always merges leaders, never arbitrary nodes.

Step 2: Check Existing Connection

```
if (pu == pv) return;
```

If both nodes already have the same leader, they already belong to the same component. Example: Tree with root 0 and children 1, 2 → `unionBySize(1,2)` gives `pu = 0`, `pv = 0`. Already connected, no work needed.

Step 3: Compare Component Sizes & Merge

```
if (size[pu] < size[pv])
```

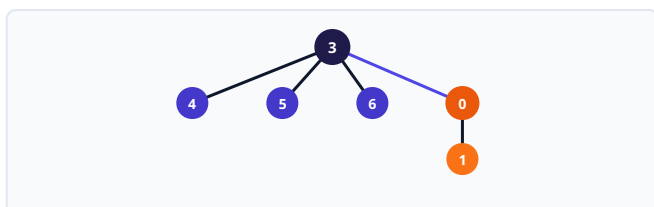
Compare the sizes of both components. Example: Component A (root 0, size 3), Component B (root 3, size 2). Comparison: `3 < 2` → False, execute `else` block.

Case 1: First Component is Smaller

```
parent[pu] = pv; size[pv] += size[pu];
```

Suppose Component A (`0 ← 1`, Size = 2) and Component B (Root 3 with children 4, 5, 6, Size = 4).

Merge: `unionBySize(1, 5)` → Leaders `pu = 0`, `pv = 3`. Since `2 < 4`, attach leader 0 under leader 3 (`parent[0] = 3`).



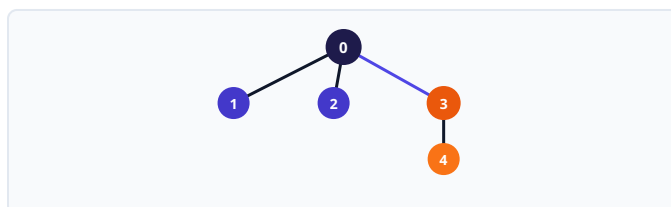
Only leader 0 changes parent (`0 → 3`). Node 1 automatically moves into component 3. Updated `size[3] = 6`.

Case 2: Second Component is Smaller

```
parent[pv] = pu; size[pu] += size[pv];
```

Suppose Component A (Root 0 with children 1, 2, Size = 3) and Component B (`3 ← 4`, Size = 2).

Merge: `unionBySize(2, 4)` → Leaders `pu = 0`, `pv = 3`. Since `3 > 2`, attach leader 3 under leader 0 (`parent[3] = 0`).



Attach `3 → 0`. Updated `size[0] = 5`.

Why Union by Size?

If we always attached larger trees under smaller trees, tree height could become large (`0 ← 1 ← 2 ← 3 ← 4 ← 5 ← 6`), making finds take $O(n)$ time. Attaching smaller under larger keeps trees shallow, guaranteeing fast constant-time operations.

7. Complete Dry Run

Initially: Node: 0 1 2 3 | Parent: 0 1 2 3 | Size: 1 1 1 1

Step 1: `unionBySize(0,1)`

Leaders: `pu=0, pv=1` (Equal). Execute: `parent[1]=0;`
`size[0]=2;`

Parent	0	0	2	3
Size	2	1	1	1

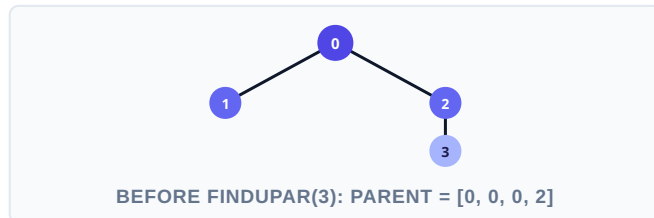
Step 2: `unionBySize(2,3)`

Leaders: `pu=2, pv=3` (Equal). Execute: `parent[3]=2;`
`size[2]=2;`

Parent	0	0	2	2
Size	2	1	2	1

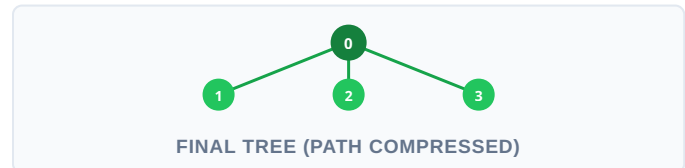
Step 3: `unionBySize(1,2)`

Leaders: `findUPar(1) = 0, findUPar(2) = 2`. Sizes: `size[0]=2, size[2]=2`. Execute `parent[2]=0; size[0]=4;`



Step 4: After calling `findUPar(3)`

Path compression updates parent array to `[0, 0, 0, 0]`.



8. Time Complexity Analysis

With **Union by Size + Path Compression**:

- `findUPar()` → $O(\alpha(N))$
- `unionBySize()` → $O(\alpha(N))$

where $\alpha(N)$ is the *Inverse Ackermann Function*, which grows so slowly that for any practical value of N , it is less than 5. Therefore, both operations are considered **almost $O(1)$** in practice.

9. API Summary Table

Function	Purpose	Time Complexity
<code>DSU(int n)</code>	Initializes n separate components	$O(N)$
<code>findUPar(node)</code>	Finds the leader (ultimate parent) of a node with path compression	$O(\alpha(N))$
<code>unionBySize(u, v)</code>	Merges two components by attaching the smaller one under the larger one	$O(\alpha(N))$

10. Key Points to Remember

- Every component has one leader (ultimate parent).
- `parent[]` stores the parent of each node.
- `size[]` stores the number of nodes in a component (valid for the leader).
- `findUPar()` finds the leader and performs path compression to flatten the tree.
- `unionBySize()` always connects the leaders, never arbitrary nodes.
- The smaller component is attached under the larger component to keep the tree shallow.
- Combining Union by Size and Path Compression makes DSU one of the fastest data structures for dynamic connectivity problems.